HACKERNOON

Claude Book: A Multi-Agent Framework for Writing Novels with Claude Code

Written by thomashoussin | Published 2026/01/27
Tech Story Tags: ai-fiction | claude-code | mistral-ai | ai-slop | perplexity-analysis | ai-narrative-coherence | claude-book | hackernoon-top-story

TLDR

Using subagents and perplexity gates to maintain consistency and add texture to AI-generated fiction.

The Problem with AI-Generated Fiction

Large language models can write, a lot. But they have two persistent problems:

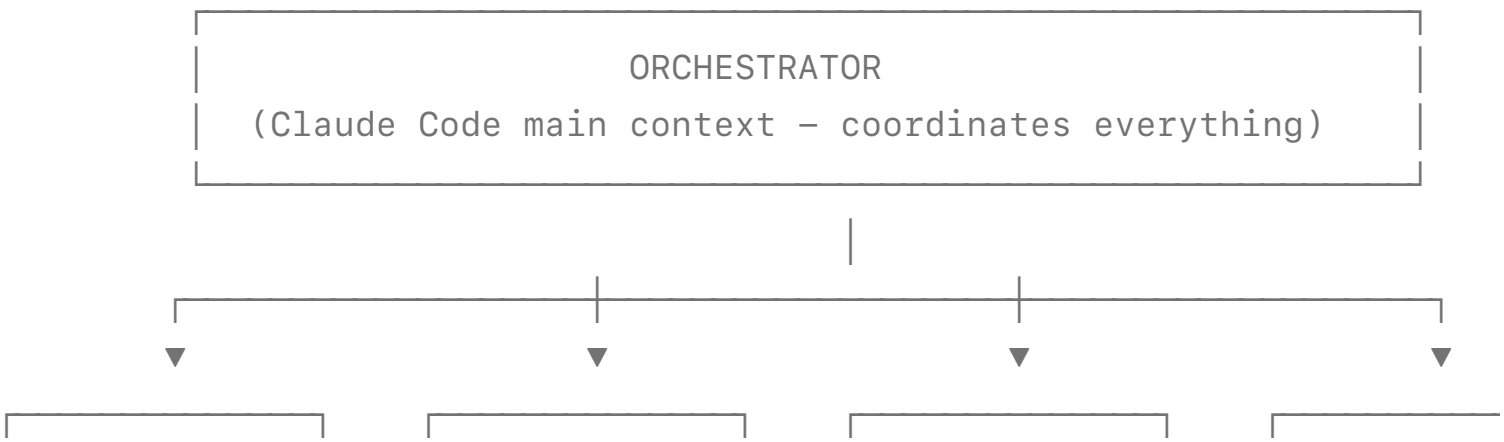
1. **Coherence drift**: Over long outputs, they forget character traits, timeline details, and plot threads.
2. **The AI Slope**: Their output slides toward predictable patterns: the statistical average. The text takes the "well-trodden" path.

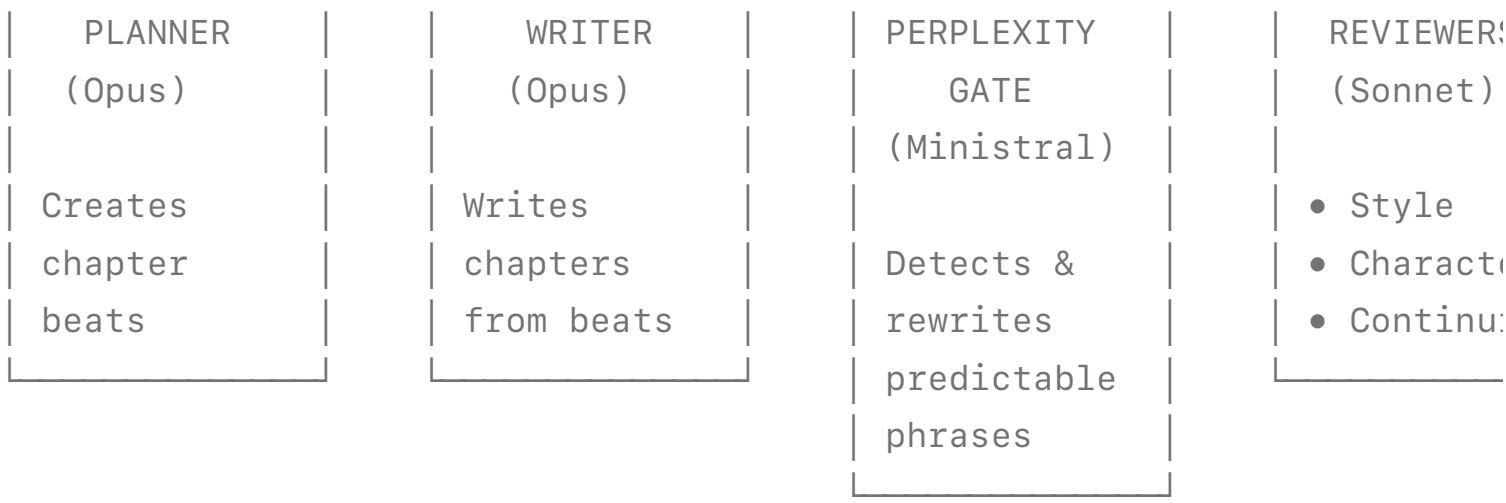
I wanted to work on both. The result is what I call **Claude Book**, a framework that uses Claude Code as an orchestrated writing system, with quality and consistency gates, including a local perplexity-based quality gate.

The proof of concept: a complete 18-chapter novel in French, in the style of Enid Blyton's *Famous Five* series (*Le Club des Cinq*) and a second 3000-word chapter in English.

Architecture Overview

The framework follows an orchestrator-worker pattern with specialized agents:





Each agent has a single responsibility. The orchestrator injects context, validates gates before proceeding, and corrects minor continuity errors.

Short Presentation

Before diving into each component, here's how the framework organizes its files. The framework has 4 main folders: bible, state, story, timeline :

- bible: structured reference that never changes during generation
- state: the current situation of the story
- story: the actual story, synopsis, plan and chapters
- timeline: detailed timeline, updated after each chapter

The Bible: Your Source of Truth

Before writing begins, you create **a bible**, a structured reference that never changes during text generation.

Two skills can help you:

- The *book-analyzer* skill can extract this automatically from source books. This gives you quantified style rules: average sentence length, dialogue tag frequency, vocabulary constraints, and explicit forbidden elements.
- The *bible-merger* can then merge several analyses extracted from several source books of the genre you want to imitate.

The bible is an important part of the framework: it defines the style and forbidden elements. It details the character, their voices, their speech patterns, dialogue examples, etc. That's what

will give them some personality and texture when writing. An incomplete or low quality bible will give you boring text.

State Management: The Key to Coherence

The state tracks the current situation of the story: character locations, inventory, knowledge, and relationships. After each validated chapter, the *state-updater* agent extracts changes and creates a new versioned snapshot.

The directory structure uses a symlink pattern:

```
├── state/
│   ├── current/        # Symlink → latest chapter state
│   ├── chapter-01/     # Archived state after chapter 1
│   ├── chapter-02/     # Archived state after chapter 2
│   └── template/       # State file templates
```

The *current/* symlink always points to the latest validated state. This means every agent reads from `state/current/` without needing to know which chapter was last completed, no path changes, no version tracking in prompts.

Why this matters: When writing chapter 15, the model has access to exactly what happened in chapters 1-14, without needing 100K tokens of context.

The Agent Workflow

Here's how a chapter gets written:

1. When a new chapter is in plan mode, the prompt is pretty simple: write a chapter, following the workflow. Exit plan mode.
2. The planner then writes chapter beats. Beware: the quality of the beats depends on your synopsis; the more detailed, the better they will be. And if the beats are lacking, the writer will "fill" the chapter with whatever it has...
3. The writer agent generates a full chapter from the beats and writes the chapter in the draft folder.
4. Perplexity skill checks and flags the sentences to be rewritten.
5. Finally, subagents run to check in parallel for style, character, and continuity (validation gate).
6. When everything is validated, the chapter is moved to the story folder, and state-agents update

the current state.

Each reviewer agent has strict boundaries. The subagents are in charge of validation; they don't write the novel themselves. The coordinator and writer agent are the ones to write (the coordinator when calling perplexity skill, the writer if the chapter has to be rewritten).

The Agents in Action: Real Examples

Style Linter: Enforcing Tone

The style bible includes explicit constraints under "Forbidden elements": for example, in *Famous Five*, kids' book: *"Death of characters (even villains get arrested, not killed)"* and *"Graphic violence or detailed injuries."*

When describing an old lighthouse keeper's fate, the chapter beats included:

"The keeper, Mr. Le Goff, died during a terrible storm. 'They say he fell down the stairs in the middle of the night, while the waves were smashing against the rocks. They only found his body two days later.'"

The style linter flagged this: explicit death, a body being found were too dark for the target register. The rewritten version:

"The keeper, Mr. Le Goff, disappeared during the storm that year. A terrible night, with waves as tall as houses. The poor man was never seen again."

"Disappeared" or "Never seen again". That's Blyton's register, mystery and pathos without graphic detail, and a more appropriate tone.

Character Linter & Continuity Linter

The idea behind these two is very simple: it just checks that what's written is consistent with the bible and with the state file.

This is where having a state file with situation, inventory, and knowledge at the end of the chapter is interesting: at the end of the chapter, you have a dark night without the moon. At the beginning of the next one, the moonlight illuminates the landscape. Or an item mysteriously duplicating. This is the kind of error that gets flagged by this reviewer. It even flagged a mistake I made in the synopsis, and that was creating an inconsistency in the plot.

Style Checker

The idea of the style checker is the same, checking consistency with the style section in the bible. I added a simple script `style_checker.py`, that computes some stuff that doesn't

need a language model:

- **AI-signal words**, terms that LLMs overuse: "delve", "showcasing", "boasts", "underscores", intricate", "realm", "groundbreaking"
- **Dialogue ratio** computes the word ratio (dialogue being matched by quoted text)
- **Forbidden dialogue tags**, if you want to have any
- **Repetition analysis**, word frequency per page, flagging when a term appears more than 3× per 250 words (could be improved to use a sliding window btw)
- **Quote style**, catching French guillemets « » when writing in English

These are cheap checks, no GPU required. The script generates a report that the style-linter agent uses alongside the more intelligent checking done by the LLM (POV consistency, tense consistency, etc.)
The Perplexity Gate: Matching Human Prose
This is where it gets interesting, and where I need to be precise about what we're actually measuring.

Perplexity measures how "surprised" a language model is by text. Low perplexity means the model was very confident about the next token, and the text took the most predictable path. High perplexity means the text was harder to predict.

Important: This is NOT an AI detector. It's a diagnostic tool for "boring" text.

AI-generated text can have a phenomenon called the **AI Slope**: generated text tends to slide toward the statistical average. Every sentence takes the path of least resistance. No errors, but no friction either. The result is text that's too "smooth": predictable phrasing, uniform rhythm, lack of surprising word choices.

But **human text can also have low perplexity**. Short dialogues, common expressions, and simple declarative sentences. These are naturally predictable, regardless of who wrote them.

So what are we really trying to detect? Not "AI vs Human" but "flat text vs textured text".

The goal is to add variety and texture that makes prose feel alive. To do that, I built a local analysis pipeline using the latest *Ministral 8B* to measure sentence-level perplexity. The script applies multiple diagnostic criteria:

Detection Criteria

Criterion	Threshold	What it catches
Low perplexity	PPL < 22	Individual sentences taking the predictable path
Low std windows	σ < 14 over 14 sentences	Passages with uniform perplexity: no surprises
Adjacent low blocks	4+ consecutive sentences with PPL < 30	Extended stretches without friction
Low PPL density	>30% of window below PPL 25	Cumulative "boredom" signal
Forbidden words	Exact match	AI-signal vocabulary

Each criterion catches a different symptom of the AI Slope. And combined signals, a sentence that's individually predictable and sits in a low-variance window, are much stronger indicators of flat text. Depending on what you write or analyze, you may need to update the values (these are the ones I obtained based on AI-generated texts and English novels). Full script is on GitHub.

Important Caveat

These thresholds are diagnostic signals, not verdicts. The tool flags sentences that *might* be sliding toward predictable patterns, but many will be false positives:

- Short dialogue exchanges ("'Yes,' she said.")
- Common expressions and idioms
- Simple action descriptions
- Technical or factual statements

The goal is not to rewrite everything, but to be under a target (someting like 20% or 30%) ; and to rewrite the most flagged parts of the text. Not every flagged sentence needs rewriting, only the ones that feel flat in context.

The Rewriting Skill

When the perplexity gate flags suspect sentences, the *perplexity-improver* skill rewrites them using documented techniques:

- Verbalized Sampling (VS)

- Fragmentation (FR)
- Character Voice (CV)
- Rare Vocabulary (RV)
- Syntactic Inversion (SI)
- Sensory Details (SD)
- Broken Rhythm (BR)
- Cliché Subversion (CS)
- Narrative Ellipsis (NE)

About Verbalized Sampling: VS is a prompting technique from this paper that addresses "mode collapse", the tendency of aligned models to produce repetitive, homogeneous text. Two approaches:

- Ask the model to generate multiple alternative phrasings ("give me 5 ways to say this") — recovers diversity from the base model
- Directly request outputs from the tail of the probability distribution ("sample from the tails, probability < 0.10"), to force less typical alternatives

It's not easy ot use that approach when writing from the beats, but you can do it while rewriting the borring sentences. The perplexity-improver skill does that with the second approach: it asks the model to rewrite flagged sentences with phrasings that are deliberately less predictable.

Some examples; these ones come from an experiment in English: a short story generated in one shot by Sonnet 4.5 from a brief synopsis. These are extracts of the report written by Opus 4.5 after the rewriting.

Flagged reason: Adjacent low-perplexity block
Original:
A couple walked by with a golden retriever puppy. A woman in Lululemon tights jogged past, AirPods in, face serene with whatever podcast was convincing her that everything would work out. The marine air was creeping in now, that particular chill that made you regret not bringing a jacket even though it had been seventy degrees an hour ago.
Rewritten:

A golden retriever puppy tugged its owners past, all tongue and chaos. Behind them, a woman in Lululemon moved through her jog like a woman in a commercial—AirPods in, expression arranged, probably absorbing some podcast about manifesting abundance. The marine layer announced itself then, that creeping Bay chill that punished everyone who'd trusted the seventy-degree afternoon.
Techniques used: VS ("all tongue and chaos", "like a woman in a commercial", "manifesting abundance", "punished everyone who'd trusted") + SD + RV ("marine layer announced itself")

Another example with dialogue:
Flagged reason: Adjacent low-perplexity block
Original:
Sarah returned with two plastic cups, condensation already forming on the sides. 'Okay,' she said, settling back into her chair with the air of someone beginning an interrogation. 'So. Tell me about these dreams.' [...] 'Which means they're either about work, which would be boring and you would have just told me, or they're about Tom, which means they're interesting and you've been avoiding telling me.'
Rewritten:
Sarah set down two plastic cups, condensation already bleeding down the sides. She arranged herself in the chair with the deliberateness of someone uncapping a scalpel. 'So. These dreams.'" [...] "'Which means: one, work dreams, boring, you would've said. Or two, Tom dreams. Interesting. Which is why you vanished.'
Techniques used: VS ("bleeding down the sides", "uncapping a scalpel") + FR + CV (tightened dialogue rhythm)

One more :
Flagged reason: low_perplexity on two sentences
Original:
Look, I'm not trying to push. I'm just saying—you've been together what, three years? And every time we talk lately, there's this... I don't know. This hesitation. Like you're reading from a script you're not sure you believe anymore.
Rewritten:
Look. I'm not pushing. But you've been together three years, and lately there's this... hesitation.

In your voice. Like you're reciting lines from a show you've stopped believing in.
Techniques used: VS ("reciting lines from a show you've stopped believing in") + FR
Constraints

The skill applies guardrails:

- Meaning preservation: the rewrite must say exactly the same thing
- Natural integration: the new sentence must flow with surrounding prose
- Iteration cap: max 3 rewriting loops per chapter

The goal is variety, not obfuscation. A rewrite that's technically higher-perplexity but sounds forced defeats the purpose.
Results
And at the end of the full pipeline (agents, reviewers, perplexity-improver)? You get something interesting; the bible is enforced, and the prose has texture. Varied sentence rhythms. Unpredictable word choices. It doesn't slide toward the statistical average.

Is this "undetectable"? Probably not. AI detectors like ZeroGPT often return "Likely human written" on the final text, but that's a side
effect of adding variety, not the goal. The detectors flag the same thing the perplexity gate measures: predictable patterns (and they probably flag other tells not implemented in this framework). But the real metric is : **write text that doesn't feel flat.** And I did have some nice suprises when writing with the framework.
What's Next?
The framework is MIT-licensed and available on GitHub; feel free to contribute. Ideas I'm exploring:

- Genre adaptation: Improve the bible template, and specifics for genre (thriller, sci-fi patterns, etc.)
- Additional diagnostic signals: Word frequency analysis, sentence rhythm metrics (burstiness, Fano factor), and style fingerprinting
- Rewriting technique library: Documenting more techniques beyond the current nine, with examples for each genre

And if you write something with it, I'd love to hear about it.

Written by thomashoussin | Banker building AI products → codecrafter.fr

Published by HackerNoon on 2026/01/27